

Containerization of Serverless Workloads and Understanding Its Cost

Aaryan Bhagat : 862468325

Soorya Saravannan : 862289961

Goal

Serverless computing offers scalability and cost efficiency but faces challenges like cold starts and vendor lock-in. Containerization provides portability and flexibility. This project compares the cost and performance of both approaches to identify their strengths and best-use scenarios.

Design

- Kubernetes Cluster consisting of multiple nodes and a master node.
- Each node will have multiple pods which will run either only docker containers or knative serverless applications.
- The nodes will be maximally utilized using vertical scaling.

Implementation

- Creating a bare metal kubernetes cluster on cloud lab.
- Since serverless and containers have their own pros and cons, different tasks were made to test each one of their strengths and weaknesses.
- Inference code is packaged into a Docker container with all dependencies.
- Three Docker containers are manually run to simulate container level execution.
- Inference container is deployed as a Knative service on Kubernetes.
- Inference script logs CPU, memory, and network metrics during execution
- Comparisons were made between the two approaches

Demo

[Load testing - serverless](#)

[Docker](#)

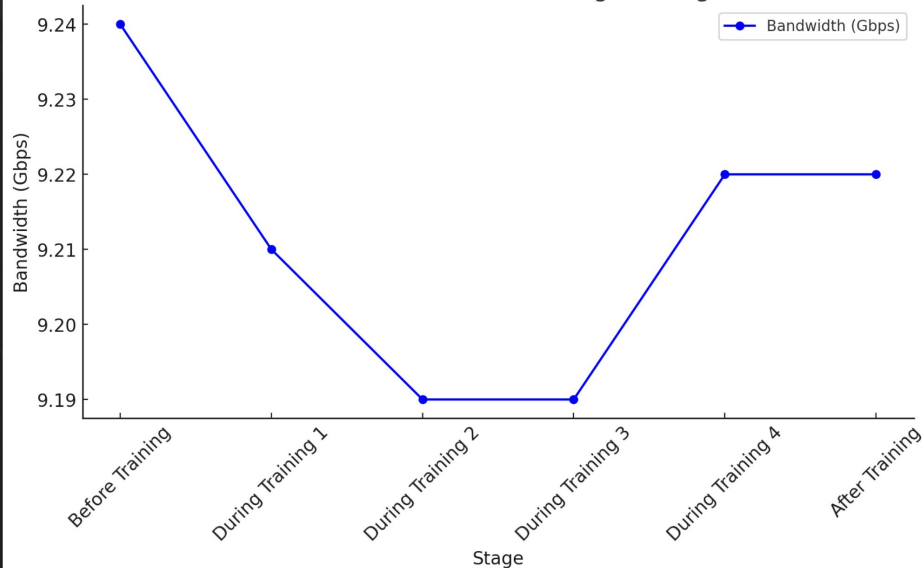
[Knative](#)

[Codebase](#)

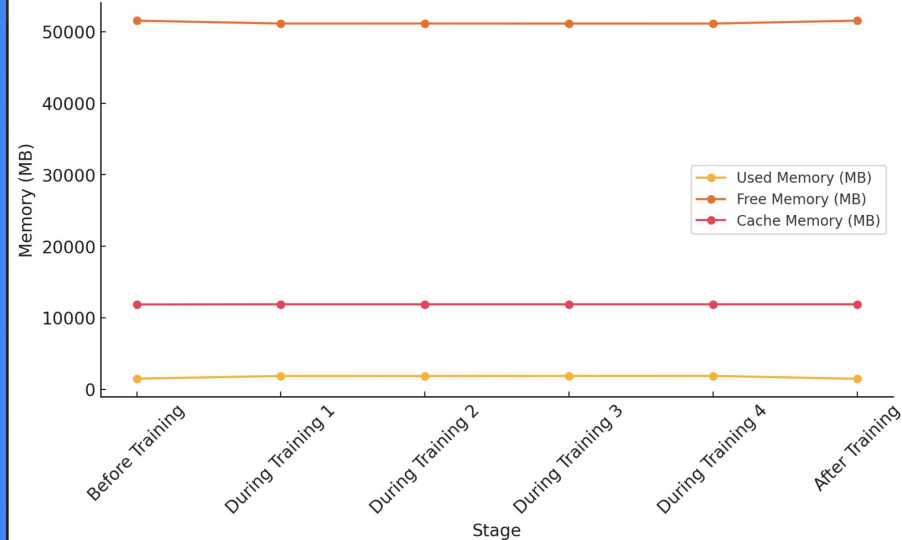


Some results

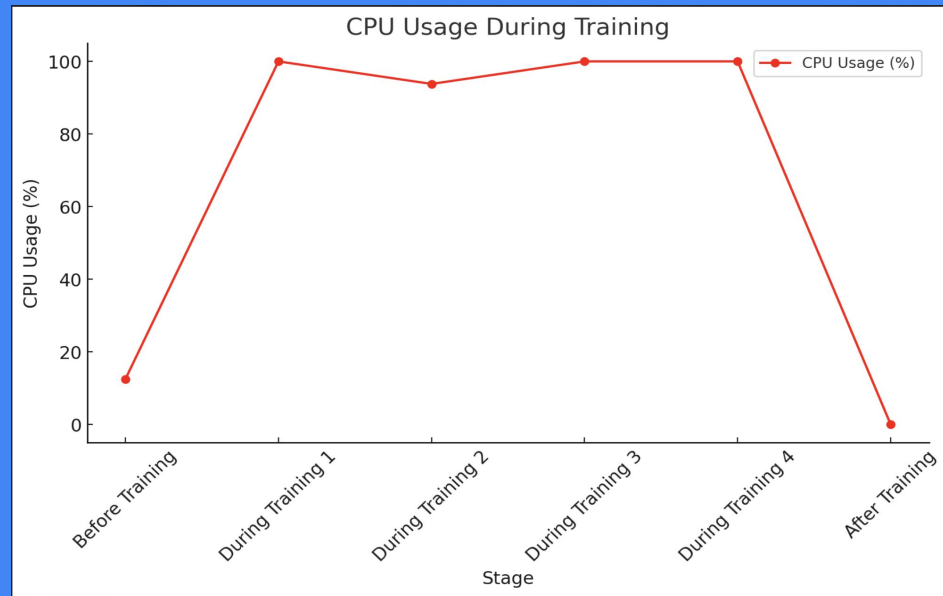
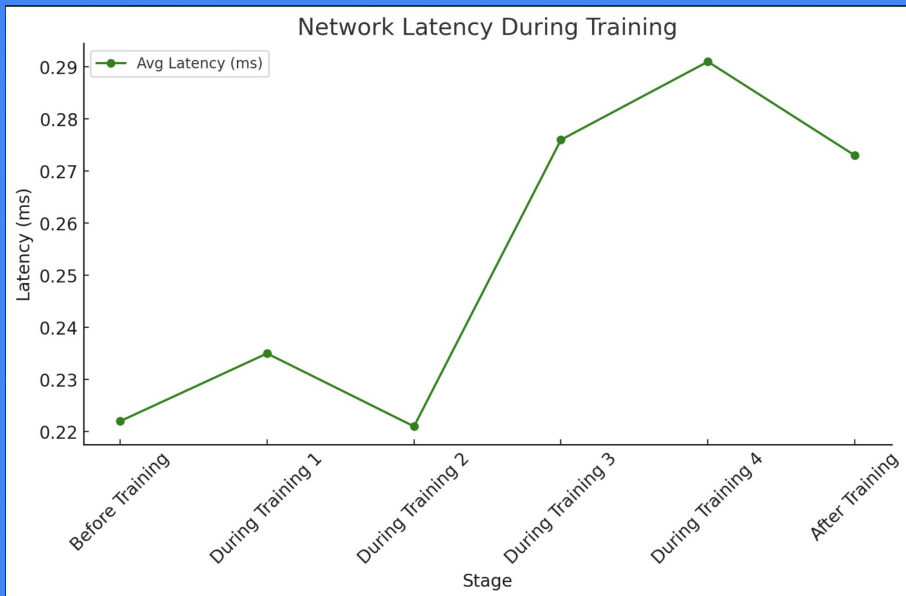
Network Bandwidth During Training



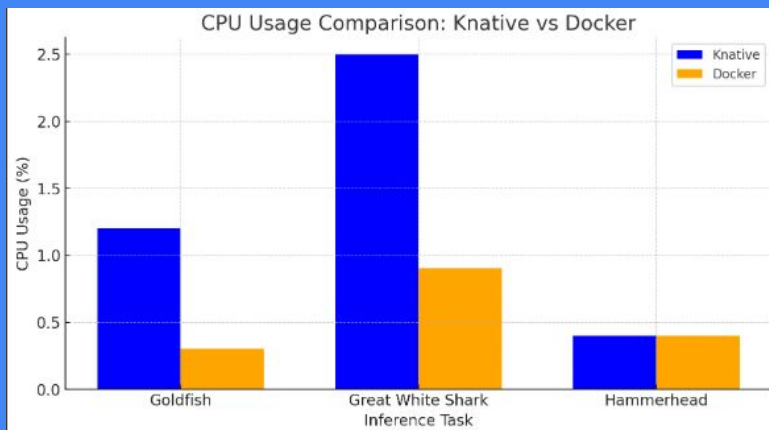
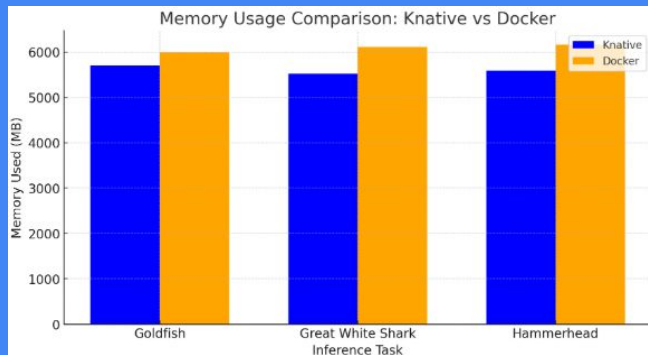
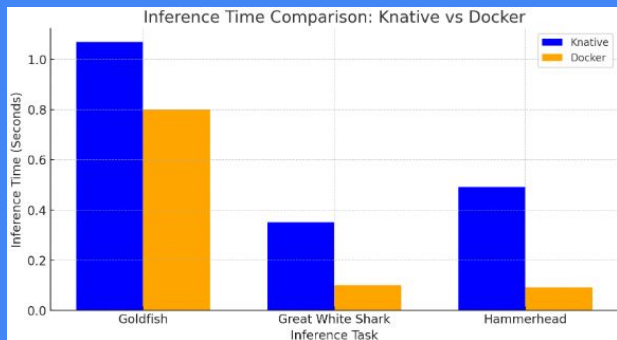
Memory Usage During Training



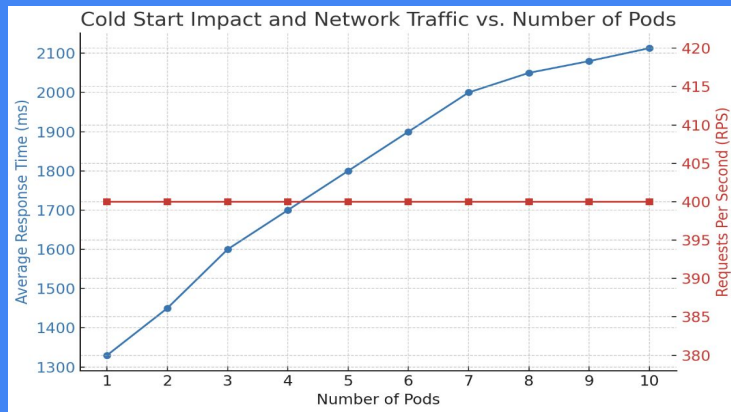
Some results



Some results



Results of Autoscaling Serverless



```
PodCIDRs: 192.168.2.0/24
Non-terminated Pods: (7 in total)
  Namespace      Name
  -----
  knative-serving net-kourier-controller-75774fd8d5-mlnc
  kourier-system  3scale-kourier-gateway-58c8cb46c6-215jl
  kube-system     calico-node-r9fkt
  kube-system     kube-proxy-h5jtq
  kube-system     metrics-server-75bf97fcc9-7fz9w
  metallb-system  controller-5b5b575d55-z6wnd
  metallb-system  speaker-l9w4r
Allocated resources:
  (Total limits may be over 1000 Request Units)
```

Namespace	Name	CPU Requests	CPU Limits	Memory Requests	Memory Limits	Age
knative-serving	net-kourier-controller-75774fd8d5-mlnc	200m (0%)	500m (0%)	200Mi (0%)	500Mi (0%)	7h38m
kourier-system	3scale-kourier-gateway-58c8cb46c6-215jl	200m (0%)	500m (0%)	200Mi (0%)	500Mi (0%)	7h38m
kube-system	calico-node-r9fkt	250m (0%)	0 (0%)	0 (0%)	0 (0%)	8h
kube-system	kube-proxy-h5jtq	0 (0%)	0 (0%)	0 (0%)	0 (0%)	8h
kube-system	metrics-server-75bf97fcc9-7fz9w	100m (0%)	0 (0%)	200Mi (0%)	0 (0%)	162m
metallb-system	controller-5b5b575d55-z6wnd	0 (0%)	0 (0%)	0 (0%)	0 (0%)	150m
metallb-system	speaker-l9w4r	0 (0%)	0 (0%)	0 (0%)	0 (0%)	150m

Sending request to 1 flask container

[2025-03-21 03:02:25.690] k8sworker/INFO/locust.runners: Ramping to 100000 users at a rate of 100.00 per second

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	4	0 (0.00%)	1353	1036	1827	1200	0.00	0.00
Aggregated		4	0 (0.00%)	1353	1036	1827	1200	0.00	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	9	0 (0.00%)	1880	1036	2822	2000	0.00	0.00
Aggregated		9	0 (0.00%)	1880	1036	2822	2000	0.00	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	16	0 (0.00%)	3256	1036	5907	2300	2.25	0.00
Aggregated		16	0 (0.00%)	3256	1036	5907	2300	2.25	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	20	0 (0.00%)	4091	1036	7885	4300	2.33	0.00
Aggregated		20	0 (0.00%)	4091	1036	7885	4300	2.33	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	25	0 (0.00%)	4925	1036	9095	4800	2.43	0.00
Aggregated		25	0 (0.00%)	4925	1036	9095	4800	2.43	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	31	0 (0.00%)	6137	1036	11918	5900	2.50	0.00
Aggregated		31	0 (0.00%)	6137	1036	11918	5900	2.50	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	36	0 (0.00%)	7051	1036	13715	6400	2.60	0.00
Aggregated		36	0 (0.00%)	7051	1036	13715	6400	2.60	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	41	0 (0.00%)	8003	1036	15416	8000	2.60	0.00
Aggregated		41	0 (0.00%)	8003	1036	15416	8000	2.60	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	48	0 (0.00%)	9273	1036	17512	9100	2.60	0.00
Aggregated		48	0 (0.00%)	9273	1036	17512	9100	2.60	0.00

Every 2.0s: kubectl describe node k8sworker1.example.net | grep -B 20 -A 200 Requests

```
cpu: 128
ephemeral-storage: 60345081347
hugepages-1Gi: 0
hugepages-2Mi: 0
memory: 263451944Ki
pods: 110
```

System Info:

```
Machine ID: 7c52626e573b484f813fed3055eb9193
System UUID: 4c4c4544-0033-3510-8034-b1c04f324733
Boot ID: 8e51577a-edeb-4784-836f-28ed4feb77a5
Kernel Version: 5.15.0-131-generic
OS Image: Ubuntu 22.04.2 LTS
Operating System: Linux
Architecture: amd64
Container Runtime Version: containerd://1.7.24
Kubelet Version: v1.28.15
Kube-Proxy Version: 192.168.2.0/24
```

PodCIDRs: 192.168.2.0/24

PodCIDRs: 192.168.2.0/24

Non-terminated Pods: (8 in total)

Namespace

Name

```
default flask-random-file-generator-7f68d5b9f6-rghz 1m (0%) 3m (0%) 32Mi (0%) 64Mi (0%) 6m37s
default ml-inference-docker-6c6978ddc4-ncq4x 0 (0%) 0 (0%) 0 (0%) 0 (0%) 6m24m
keda keda-admission-7dc48f44c4-lmtd4 100m (0%) 1 (0%) 100Mi (0%) 1000Mi (0%) 132m
keda keda-metrics-api-server-758dc4f488-lftw 100m (0%) 1 (0%) 100Mi (0%) 1000Mi (0%) 132m
keda keda-operator-85bcb668d-ddhg9 100m (0%) 1 (0%) 100Mi (0%) 1000Mi (0%) 132m
kubernetes kube-system calico-node-cf8sz 250m (0%) 0 (0%) 0 (0%) 0 (0%) 6m47m
kubernetes kube-system kube-proxy-tbz1 0 (0%) 0 (0%) 0 (0%) 0 (0%) 6h51m
kubernetes kube-system metrics-server-75bf97fcc9-47tq6 100m (0%) 0 (0%) 200Mi (0%) 0 (0%) 6h44m
```

Allocated resources:
(Total limits may be over 100 percent, i.e., overcommitted.)

Resource

Requests

Limits

cpu 651m (9%) 3003m (2%)

memory 532Mi (0%) 3664Mi (1%)

ephemeral-storage 0 (0%) 0 (0%)

hugepages-1Gi 0 (0%) 0 (0%)

hugepages-2Mi 0 (0%) 0 (0%)

Events: <none>

Sending request to 3 flask containers

```
}
root@k8smaster: /users/abhag017/scale-docker# locust -f test.py --headless -u 100000 -r 100 -t 30m --html report.html
[2025-03-21 03:07:02,183] k8smaster/INFO/locust.main: Starting Locust 2.33.2
[2025-03-21 03:07:02,183] k8smaster/INFO/locust.main: Run time limit set to 1000 seconds
```

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
Aggregated		0	0(0.00%)	0	0	0	0	0.00	0.00

[2025-03-21 03:07:02,184] k8smaster/INFO/locust.runners: Ramping to 100000 users at a rate of 100.00 per second

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	10	0(0.00%)	1710	1078	1961	1800	0.00	0.00
Aggregated		10	0(0.00%)	1710	1078	1961	1800	0.00	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	28	0(0.00%)	2480	1078	3526	2400	0.00	0.00
Aggregated		28	0(0.00%)	2480	1078	3526	2400	0.00	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	45	0(0.00%)	3463	1078	5816	3100	7.00	0.00
Aggregated		45	0(0.00%)	3463	1078	5816	3100	7.00	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	63	0(0.00%)	4490	1078	7915	4600	7.33	0.00
Aggregated		63	0(0.00%)	4490	1078	7915	4600	7.33	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	82	0(0.00%)	5496	1078	9909	5600	7.50	0.00
Aggregated		82	0(0.00%)	5496	1078	9909	5600	7.50	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	100	0(0.00%)	6383	1078	11804	6000	7.90	0.00
Aggregated		100	0(0.00%)	6383	1078	11804	6000	7.90	0.00

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/	121	0(0.00%)	7446	1078	13699	7800	9.50	0.00
Aggregated		121	0(0.00%)	7446	1078	13699	7800	9.50	0.00

When requests are increased 2 containers are created in one node

```
Every 2.0s: kubectl describe node k8sworker1.example.net | grep -B 20 -A 200 Requests
```

```
cpu: 120
ephemeral-storage: 60345081347
hugepages-1Gi: 0
hugepages-2Mi: 0
memory: 263451944Ki
pods: 110
System Info:
  Machine ID: 7c52626e573b484f813fed3055eb9193
  System UUID: 4c4c4544-0033-3510-8034-b1c04f324733
  Boot ID: 8e5f57fa-e0eb-4784-898f-28edf4eb77a5
  Kernel Version: 5.15.0-131-generic
  OS Image: Ubuntu 22.04.2 LTS
  Operating System: linux
  Architecture: amd64
  Container Runtime Version: containerd://1.7.24
  Kubelet Version: v1.28.15
  Kube-Proxy Version: v1.28.15
PodCIDR: 192.168.2.0/24
PodCIDRs: 192.168.2.0/24
Non-terminated Pods: (9 in total)
  Namespace      Name
  -----
  default         flask-random-file-generator-688d9fbcfb-67rng
  default         flask-random-file-generator-688d9fbcfb-mzk6k
  default         ml-inference-docker-6c6978ddc4-ncq4x
  keda             keda-admission-7dc88f744c-tnn64
  keda             keda-metrics-api-server-758dc4f488-1ftvw
  keda             keda-operator-85bc6c668d-ddhg9
  kube-system     calico-node-crggz
  kube-system     kube-proxy-rtmzt
  kube-system     metrics-server-75bf97fcc9-47tq6
Allocated resources:
(Total limits may be over 100 percent, i.e., overcommitted.)
Resource           Requests    Limits
-----
cpu                652m (0%)  300m (2%)
memory             564Mi (0%) 3128Mi (1%)
ephemeral-storage  0 (0%)     0 (0%)
hugepages-1Gi     0 (0%)     0 (0%)
hugepages-2Mi     0 (0%)     0 (0%)
Events:            <none>
```

one is created in another node

```
Every 2.0s: kubectl describe node k8sworker2.example.net | grep -B 20 -A 200 Requests
```

```
cpu: 40
ephemeral-storage: 60345081347
hugepages-1Gi: 0
hugepages-2Mi: 0
memory: 263926772Ki
pods: 110
System Info:
  Machine ID: 46967b30ee142f8af3fc301cb012b7d
  System UUID: 4c4c4544-0031-4610-804a-c4c04f303432
  Boot ID: 54e59b62-cbcc-4158-a155-788aadcc7337
  Kernel Version: 5.15.0-131-generic
  OS Image: Ubuntu 22.04.2 LTS
  Operating System: linux
  Architecture: amd64
  Container Runtime Version: containerd://1.7.24
  Kubelet Version: v1.28.15
  Kube-Proxy Version: v1.28.15
PodCIDR: 192.168.1.0/24
PodCIDRs: 192.168.1.0/24
Non-terminated Pods: (4 in total)
  Namespace      Name
  -----
  default         flask-random-file-generator-688d9fbcfb-2hnlm
  default         python-http-server-598cb4b7d7-95fg5
  kube-system     calico-node-rj2ct
  kube-system     kube-proxy-w9rw9
Allocated resources:
(Total limits may be over 100 percent, i.e., overcommitted.)
Resource           Requests    Limits
-----
cpu                261m (0%)  53m (0%)
memory             64Mi (0%)  128Mi (0%)
ephemeral-storage  0 (0%)     0 (0%)
hugepages-1Gi     0 (0%)     0 (0%)
hugepages-2Mi     0 (0%)     0 (0%)
Events:            <none>
```

Learning from errors

- Kubernetes autoscaling, specifically the part which required to create more cloumlab nodes needed an internal integration with the cloumlab environment.
- Sadly this integration was not present, the alternate option was to create ample amount of nodes and give a custom load balancer however this will assume all the run times for the containers are already up and running so it will not intake the startup times of the containers.
- Load tests need to be efficiently designed to stress test docker containers as they have good memory and cpu cycles associated with it.

Eventual Realization

- Event based tasks serverless still holds superiority both in terms of cost and speed (inference for e.g).
- Containers can still undergo tasks having high amount of load and it needs less wrappers to check the metrics.
- Cost analysis for both of these computing beds are different
- Its easy to vertically scale a serverless than a container (our experimental realization).

Thanks!



Questions and Answers
